

SENIOR SWE INTERVIEW SERIES

Module 7: CDN

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 7 — CDN

The CDN is the first box in almost every internet-facing design. Interviewers test whether you know what it actually does (much more than “caches images”), how invalidation really works, and the cache-header contract you own as a backend engineer.

Topic 7.1 — CDN Architecture

1. Why Interviewers Ask This

“Add a CDN” is easy to say; interviewers immediately probe: what’s inside a PoP, how does a miss flow to origin, what’s tiered caching, what benefits apply to *dynamic* (uncacheable) traffic? That last one separates seniors.

2. Core Concept

A CDN is a globally distributed fleet of proxy/cache servers (**edge PoPs**) placed near users. Requests hit the nearest edge (via DNS/anycast steering — Module 5.5). Cache hit → served in ~5–30ms. Miss → fetched through the CDN’s backbone to your **origin**, cached per your headers, then served. Beyond caching: TLS termination near users, TCP/QUIC optimization, DDoS absorption, WAF, edge compute.

3. Internal Working

- **PoP internals:** L4 balancer → cache servers sharded by consistent-hash-on-URL (each object lives on 1–2 machines per PoP, not all); hot-object replication for celebrities/flash sales; RAM for hot, NVMe for warm.
- **Tiered caching:** edge miss → *regional/shield* parent PoP → origin. Shields collapse thousands of edge misses into few origin fetches.
- **Request coalescing** (a.k.a. collapsed forwarding): 10k concurrent misses for the same URL → 1 origin fetch, 9,999 waiters. The anti-thundering-herd primitive — name it.
- **Origin connectivity:** long-lived warm connections over the CDN’s private backbone (premium: dedicated fiber, e.g. Cloudflare Argo, Akamai SureRoute) — often faster than the public internet route.

4. Packet Flow Explanation

```
user(Tokyo) GET /product/42.jpg
1. DNS/anycast -> Tokyo PoP edge (RTT ~5ms)
2. TLS terminates at edge (session resumption; h3)
3. cache key = host+path(+vary) -> consistent hash -> cache node C7
4. C7: HIT (fresh) -> serve. total ~10-20ms. origin untouched.
   C7: MISS -> coalesce check -> fetch via shield PoP (Osaka)
       shield MISS -> warm backbone conn -> origin (Virginia, 150ms)
       -> store per Cache-Control at shield + edge -> serve
       first user pays ~200ms; the next million pay ~10ms.
Dynamic /api/cart (no-store): still edge-terminated TLS + warm
backbone to origin => saves 1-2 RTTs of handshakes + slow start.
```

5. ASCII Diagram

```

users ---5ms---> [EDGE PoPs x300]---(consistent hash cache nodes)
                  | miss
                  v
            [SHIELD/REGIONAL PoP] <- collapses misses
                  | miss (warm, long-lived conns, private backbone)
                  v
            [ORIGIN + LB]
hit ratios: edge ~85-95%, +shield => origin sees ~1-5% of traffic

```

6. Real Production Example

- Netflix **Open Connect**: their own CDN — OCA appliances *inside ISP networks*, filled during off-peak with predicted content; >90% of Netflix bits never touch the public internet core.
- Cloudflare: ~300+ anycast PoPs; same fleet does CDN+WAF+DNS+Workers.
- Akamai: ~4,000 deep-in-ISP locations (DNS-steered) — contrast of “few big anycast sites” vs “many deep sites” architectures is interview-worthy.

7. Advantages

- Latency: physics — content 5ms away vs 150ms; TLS/TCP handshakes near user; warmed backbone.
- Origin offload: 95%+ traffic reduction; survives flash crowds.
- Availability: serve-stale on origin down; DDoS absorbed across global capacity (Tbps).

8. Trade-offs

- Consistency: cached = possibly stale; invalidation is *the* hard problem (7.3).
- Cost per GB egress; complexity of debugging through another company’s black box.
- Cache-key subtleties (query strings, Vary) cause both leaks (wrong user’s data cached!) and fragmentation (0% hit ratio).
- Long-tail content: massive catalogs with flat popularity get poor hit ratios (why Netflix pre-positions by prediction).

9. Common Mistakes

- “CDN is only for static files” — edge TLS + backbone + h3 accelerate *dynamic APIs* too (20–40% latency cuts with zero caching).
- Ignoring the thundering-herd-on-expiry problem (no coalescing/stale-while-revalidate → origin dies at TTL boundaries).
- Caching personalized responses (missing `private` / `no-store`) — the classic “user A saw user B’s account page” incident.

10. Performance Impact

Typical numbers to quote: cache hit TTFB 10–30ms vs origin 100–400ms; origin egress ~90–99%; global p95 page load ~30–60%. Dynamic-only acceleration: ~1 to ~2 RTTs per connection + better congestion behavior on the backbone.

11. Common Interview Questions

1. Walk a request through a CDN: hit and miss paths, tiered caching, coalescing.
2. How does a CDN help uncacheable API traffic?
3. Push vs pull CDN? (Pull: cache-on-miss — the default. Push/pre-position: for predictable heavy content — Netflix, game releases.)

12. Follow-up Questions

- “How does the PoP avoid every cache node storing every object?” → consistent hashing on cache key within the PoP; hot-object replication as exception.
- “Origin is down — what can the CDN do?” → serve-stale (`stale-if-error`), static fallback pages, failover origins.
- “How would you build a mini-CDN?” → anycast or geo-DNS + NGINX cache shards + consistent hashing + coalescing + purge bus — expect this as a full design question.

13. Debugging Scenarios

- Low hit ratio: inspect cache-key config (query-string handling), `Vary` explosion (`Vary: Cookie` = per-user keys!), TTLs too short, catalog too long-tail.
- Users in one country slow: steering sending them to a far PoP (Module 5.5 debugging), or that PoP's shield/origin path degraded — use CDN per-PoP analytics + `CF-Ray` / `X-Served-By` headers to localize.

14. Best Practices

- Enable shield/tiered caching + request coalescing; keep origin behind the CDN only (block direct access — attackers bypass your WAF otherwise; allowlist CDN IPs / origin auth headers).
- Design cache keys deliberately: normalize query params, avoid `Vary: Cookie`.
- Use origin failover + serve-stale as an availability layer, not just performance.

15. Practice Questions

1. Flash sale: 5M users hit /drop at 10:00:00. Design the CDN strategy so origin sees <100 QPS. (Pre-warm, long TTL + SWR, coalescing, static-page fallback, queue page at edge.)
2. Your API p50 improved only 5% after CDN despite 90% “hit ratio.” Why might the metric mislead? (Hits on tiny assets; the slow calls are dynamic/no-store; hit ratio by request count vs by latency-weighted value.)

Topic 7.2 — Edge Servers & Edge Compute

1. Why Interviewers Ask This

“What runs at the edge?” has expanded from caches to programmable platforms (Cloudflare Workers, Lambda@Edge, Fastly Compute). Design rounds increasingly include “what would you move to the edge?”

2. Core Concept

An edge server is the CDN machine terminating the user's connection: TLS endpoint, HTTP cache, protocol translator (user $\leftrightarrow$ h3, origin $\leftrightarrow$ h1/h2), and increasingly a compute runtime (JS isolates/WASM) executing your logic at all 300 PoPs with ~0ms cold start (isolates, not containers — V8 isolate model is a differentiating detail).

3. Internal Working

- Connection duality: user-side (short RTT, TLS 1.3/h3, resumption) and origin-side (long-lived pooled h2, backbone routing).
- Edge compute constraints: milliseconds CPU budget, no local disk, KV/state via eventually-consistent global stores (Workers KV) or single-location strong stores (Durable Objects) — the consistency trade is the interview meat.
- Typical edge logic: authn token verify, A/B assignment, redirects/rewrites, bot filtering, image resizing, personalization assembly over cached fragments.

4. Packet Flow Explanation

```
user -> edge: TLS(resumed) + h3
edge worker runs (0.1-3ms):
- verify JWT signature (public key cached at edge)
- normalize URL, choose variant (A/B cookie)
- cache lookup with rewritten key
HIT: serve personalized-assembled response entirely from edge
MISS: fetch origin with coalescing; transform response; cache fragments
Result: authenticated, personalized responses with ZERO origin RTTs
for the cacheable 95%.
```

5. ASCII Diagram

```

      [ EDGE POP ]
user ->| TLS | worker(JS/WASM isolate) | cache | -> origin (only on miss)
      |      | authn, rewrite, A/B,      |      |
      |      | compose fragments          |      |
state: | KV (global, eventual) | Durable Object (single-home, strong) |
rule:  edge = latency-sensitive, logic-light; origin = source of truth
```

6. Real Production Example

- Cloudflare Workers powers auth-at-edge and full apps;
- Lambda@Edge commonly rewrites/authorizes S3/CloudFront requests (signed cookies, paywalls);
- Fastly's instant-purge + VCL made Reddit/NYT-style "cache HTML, purge on edit" architectures possible.

7. Advantages

- Sub-10ms user-perceived logic worldwide without deploying servers to 300 sites; origin offload extends to *logic*, not just bytes; blast-radius isolation (edge handles junk traffic before origin).

8. Trade-offs

- State at edge is the hard part: global-eventual or single-home-strong — no free strongly-consistent global writes (physics).
- Debugging/observability across 300 locations; vendor lock-in (proprietary runtimes); CPU/memory limits preclude heavy work.
- Another deploy surface with its own failure modes (a bad edge config = instant global outage — several famous CDN incidents were exactly this).

9. Common Mistakes

- Moving database-dependent logic to the edge (every request still round-trips to a central DB — you moved nothing but added a hop).
- Treating edge KV as a database (eventual, seconds-lag propagation).
- No canary/staged rollout for edge code — global blast radius.

10. Performance Impact

Auth verify at edge vs origin round trip: ~100–300ms for far users. Fragment assembly at edge can make personalized pages ~fully cacheable (hit ratios 30% → 90%). Numbers like these justify the architecture in interviews.

11. Common Interview Questions

1. What logic would you move to the edge, and what must stay at origin?
2. How do Workers achieve ~0ms cold starts vs Lambda's? (V8 isolates sharing a process vs microVM per function.)
3. How do you do personalization without killing cache hit ratio? (Edge composition: cache shared fragments, inject user bits at edge.)

12. Follow-up Questions

- “Where do you store a rate-limit counter for edge enforcement?” → per-PoP approximate + async global sync, or Durable-Object-style single-home per key — accuracy vs latency again.
- “How do you deploy edge code safely?” → versioned, percentage rollout by PoP/req-hash, instant rollback, config-as-code review (cite CDN config-push outages as motivation).

13. Debugging Scenarios

- Edge worker latency spikes in one region only → that PoP overloaded or its KV replica lagging; check per-PoP metrics, **CF-Ray** decoding.
- Auth randomly fails for ~1 min after key rotation → edge public-key cache TTL; overlap keys (kid) during rotation.

14. Best Practices

- Edge for: verify (not issue) tokens, route, rewrite, compose, filter. Origin for: transactions, source-of-truth writes.
- Keep edge logic dependency-free on the hot path; strict CPU budgets; staged global rollouts.

15. Practice Questions

1. Design edge-based payroll: JWT in cookie, verified at edge, cached article HTML, entitlement changes must apply <60s. (Edge verify + short-TTL entitlement claims or KV check with 60s TTL; purge on change.)
2. Which of these belongs at edge: password check, image resize, cart checkout, geo-blocking, feed ranking? Justify each in one line.

Topic 7.3 — Cache Invalidation

1. Why Interviewers Ask This

“There are only two hard things in computer science...” — interviewers use invalidation to test whether you can reason about consistency windows, purge mechanics at 300 PoPs, and design content addressing that sidesteps the problem entirely.

2. Core Concept

Three strategies, in order of preference: 1. **Immutable + versioned URLs** (cache busting): `app.3f2a1c.js`, TTL=1 year, `immutable`. New deploy = new URL. *Never invalidate; change the pointer* (the HTML referencing it has short TTL). The gold standard for static assets. 2. **TTL expiry**: passive staleness bound; pair with `stale-while-revalidate` for smooth refresh. 3. **Active purge**: tell the CDN to drop keys — by exact URL, prefix, or **surrogate keys/cache tags** (tag every response with entity IDs: `product-42`; on product update, purge tag `product-42` → all pages containing it vanish everywhere). This is the industry answer for HTML/API caching.

3. Internal Working

Purge propagation: API call → CDN control plane → broadcast to all PoPs (pub/sub over their backbone) → each PoP tombstones/deletes matching keys. Fastly's famous ~150ms global purge = purge messages race over dedicated infrastructure; many CDNs take seconds-minutes for path/wildcard purges (metadata scans). Soft purge marks stale (revalidate on next hit) vs hard purge deletes (next hit = full miss — herd risk!).

4. Packet Flow Explanation

```
CMS: editor saves article 87
1. origin emits purge: POST cdn/purge {surrogate-key: article-87}
2. control plane fans out to 300 PoPs (~150ms..s)
3. edge marks all objects tagged article-87 STALE (soft purge)
4. next request per PoP: serve-stale + async revalidate (SWR) ->
   conditional GET w/ ETag -> 304 or new body -> cache refreshed
consistency window: purge-latency + per-PoP revalidation ≈ seconds
vs TTL-only: window = full TTL (minutes/hours)
```

5. ASCII Diagram

strategy	consistency window	origin load	use for
versioned URL	0 (new URL)	deploy-time only	JS/CSS/images
TTL(+SWR)	<= TTL	smooth	semi-static, APIs
tag purge	~seconds	burst on purge	HTML, catalogs
no-store	0 (never cached)	100%	personal/secure
anti-herd on purge: SOFT purge + SWR + coalescing (never a hot key without a warm plan)			hard-purge

6. Real Production Example

- News sites (NYT-class): cache full article HTML with surrogate keys; editor hits save → tag purge → world sees the correction in seconds; origin also survives breaking-news traffic entirely from cache.
- E-commerce: product pages tagged by SKU; price change purges thousands of listing/detail pages via one tag.
- Every modern frontend build pipeline: hashed filenames + far-future immutable (the reason you can set TTL=1yr fearlessly).

7. Advantages

Versioning: perfect consistency + perfect cacheability simultaneously (dodges the CAP-flavored trade). Tag purge: near-real-time correctness with hit ratios intact. SWR: users never wait on revalidation.

8. Trade-offs

- Purge is a *distributed delete* — eventual by nature; windows exist; failed purge to one PoP = long-tail staleness (idempotent re-purge + TTL backstop!).
- Tag systems need discipline: every response must emit correct tags (miss one → permanent staleness bug).
- Hard purge of hot keys = self-DDoS at origin (thundering revalidation).
- Wildcard purges are slow and often quota-limited.

9. Common Mistakes

- Purge-as-primary-consistency with no TTL backstop (“we purge, so TTL=∞” → one lost purge = stale forever).
- Cache-busting via query string (`?v=2`) — some caches ignore query strings by config; path-based hashes are safer.
- Purging then immediately mass-warming manually (self-herd) instead of SWR/soft purge.
- Forgetting browser caches: CDN purge doesn’t touch what’s already in users’ browsers — only short browser TTL or versioned URLs handle that (KEY distinction: you can purge your CDN, you can *never* purge browsers).

10. Performance Impact

Versioned assets: ~100% hit ratio, zero consistency cost. Tag-purge architectures let you cache previously “uncacheable” HTML → origin ~90% with seconds-level freshness. Quantify the browser-cache caveat: users mid-session keep old JS until reload — plan API backward compatibility across one version.

11. Common Interview Questions

1. How do you invalidate across 300 PoPs, and what's the realistic consistency window?
2. Design caching for a news site where corrections must appear in <10s.
3. Why are versioned URLs superior for static assets? What still needs purging? (The short-TTL HTML pointer.)

12. Follow-up Questions

- “Soft vs hard purge?” → stale-mark + revalidate vs delete; herd implications.
- “How do surrogate keys work internally?” → PoPs maintain tag → keys index; purge-by-tag walks the index (that upkeep is why tags cost extra/have limits).
- “How do you invalidate what's in users' browsers?” → you can't; version the URL or keep browser TTL short + ETag revalidation.

13. Debugging Scenarios

- “Purged but users still see old content”: browser cache (no CDN involvement), other cache layers (service worker!, corporate proxy), purge succeeded on tag but response wasn't tagged, or a second CDN/shield layer retained it. Check `Age`, `X-Cache`, `CF-Cache-Status` headers hop by hop.
- Stale for exactly some users: one PoP missed the purge → re-purge (idempotent), verify with per-PoP debug requests.

14. Best Practices

- Static: hashed URLs + `max-age=31536000, immutable`. HTML/APIs: short TTL + SWR + tag purge. Always a TTL backstop under purge-based systems.
- Emit surrogate keys from the data layer automatically (every entity render adds its tag) — humans forget.
- Purge pipeline: async, retried, idempotent, monitored (purge lag as an SLO).

15. Practice Questions

1. Product catalog: 10M pages, price updates 100/s, freshness SLO 30s, origin can take 500 QPS. Design TTL/purge/SWR mix and compute origin revalidation load.
2. A bad deploy shipped broken JS with `max-age=1yr, immutable`. Walk the recovery. (You can't purge browsers → ship new hashed filename in new HTML + purge the *HTML*; incident lesson: HTML TTL must stay short.)

Topic 7.4 — Cache Headers

1. Why Interviewers Ask This

Cache headers are the *contract you personally write* as a backend engineer — misusing them causes both outages (private data cached publicly) and waste (0% hit ratio). Interviewers quiz the directives because they're checkable knowledge with real consequences.

2. Core Concept

Cache-Control directives that matter: - **max-age=N** (freshness, seconds; browser+CDN), **s-maxage=N** (shared caches only — CDN gets a longer leash than browsers: the pro move). - **public** / **private** (CDN may store vs browser-only) / **no-store** (nowhere, ever) / **no-cache** (store but **revalidate every use** — misleading name, classic trap). - **stale-while-revalidate=N** (serve stale, refresh async), **stale-if-error=N** (serve stale on origin failure), **immutable** (skip revalidation even on reload). - Validators: **ETag** (+ **If-None-Match**) and **Last-Modified** (+ **If-Modified-Since**) → **304 Not Modified** saves bytes, not the round trip. - **Vary: <header>** — cache key extension; powerful and dangerous. - **Age** — how long the object sat in cache (your debugging friend).

3. Internal Working

Freshness algorithm every cache runs: **age < (s-maxage || max-age)** → fresh, serve. Expired → if SWR window: serve stale + async conditional GET; else block on revalidation: send **If-None-Match: <etag>** → origin replies **304** (headers only, cheap) or **200** (new body). **Vary: Accept-Encoding** splits keys per encoding (fine); **Vary: Cookie** splits per unique cookie string (catastrophic fragmentation ≈ no caching, or worse, session-keyed leaks if partially configured).

4. Packet Flow Explanation

```
GET /api/products (edge)
cached copy: Age: 45, Cache-Control: s-maxage=60, SWR=300, ETag:"v18"
t=45s  fresh -> serve (0 origin)
t=70s  stale but in SWR -> serve stale INSTANTLY + background:
      GET origin If-None-Match:"v18" -> 304 -> reset Age. user never waited.
t=400s past SWR -> blocking revalidate (first user pays origin RTT)
origin down at t=70..? -> stale-if-error=86400 keeps serving. availability!
```

5. ASCII Diagram

```
decision tree you should recite:
personal/secure?  -> Cache-Control: no-store (or private if browser ok)
hashed static?    -> public, max-age=31536000, immutable
HTML shell?       -> public, max-age=0/60, s-maxage=300, SWR, tag-purged
API list/detail?  -> private or s-maxage=30-300 + SWR + ETag
always: ETag for revalidation; s-maxage to give CDN != browser TTLs
no-cache = "revalidate always", NOT "don't cache" <- interview trap
```

6. Real Production Example

The recurring industry incident: an API returns **Set-Cookie** and **public, s-maxage** → CDN caches one user's session/page for everyone (real incidents at major sites; CDNs now refuse to cache responses bearing Set-Cookie by default — know that default). Opposite failure: **Vary: Cookie** on a marketing site silently reduced hit ratio to ~0 for years — pure money burned.

7. Advantages

Headers give you *declarative*, per-resource control honored by browsers, CDNs, and proxies uniformly — one contract, three cache tiers. SWR/SIE convert caching into an availability mechanism.

8. Trade-offs

- The vocabulary is subtle (**no-cache** vs **no-store**; **max-age** vs **s-maxage**) and mistakes fail silently (over-caching = correctness bug; under-caching = perf/cost bug).

- ETag pitfalls: default file-based ETags differ per server instance (inode-based) → fleet behind LB never 304s (Nginx/Apache config detail that wins points).

9. Common Mistakes

- `no-cache` used intending “never store” (that’s `no-store`).
- Missing `private` on authenticated responses; `Set-Cookie` on cacheable ones.
- Same TTL for browser and CDN (can’t purge browsers! browser TTL should be $\leq$ CDN’s).
- `Vary: User-Agent` (thousands of variants) or `Vary: Cookie`; strong ETags from per-server state.

10. Performance Impact

`304` responses: ~200–500 bytes vs full body — but still cost the RTT; that’s why `immutable` (skip revalidation entirely) exists for hashed assets. SWR turns p99 revalidation stalls into background work. `stale-if-error` = free availability nine(s) for read-heavy content.

11. Common Interview Questions

1. `no-cache` vs `no-store` vs `private` vs `must-revalidate` — precise semantics.
2. Design headers for: hashed JS, HTML page, personalized API, public API. (The decision tree above.)
3. How does conditional revalidation work end-to-end (ETag/304)?

12. Follow-up Questions

- “Why s-maxage at all?” → CDN is purgeable/controllable, browsers aren’t → give CDN long, browser short.
- “Strong vs weak ETags?” → byte-identical vs semantically-equivalent; weak (`W/"..."`) can’t be used for ranges; generate from content hash, not inode.
- “What does `must-revalidate` add?” → forbids serving stale beyond expiry even on origin errors (strict correctness content: finance/legal).

13. Debugging Scenarios

- Hit ratio ~0: check `Vary` explosion, `Set-Cookie` on responses, `Authorization` header presence (caches won’t store unless `public` explicitly), per-server ETags breaking 304 chains, query-string cache-key config.
- Users see stale after fix deployed: `Age` header shows CDN copy age; if Age small but content old → browser cache → your browser TTL was too long; ship versioned URL.

14. Best Practices

- Set Cache-Control on every response deliberately (framework defaults are accidents waiting).
- Split TTLs: browser short (`max-age=60`) / CDN long (`s-maxage=3600`) + tag purge + SWR + stale-if-error.
- Content-hash ETags; audit hit ratio per route as a recurring perf review.

15. Practice Questions

- Write exact Cache-Control for: (a) `/static/app.9f3c.js`, (b) `/`, (c) `/api/me`, (d) `/api/products?page=2`. Then say which are purgeable, which need versioning, and each one's staleness window.
- Your `/api/config` should be cached 5 min at CDN, never stale in browsers beyond 30s, survive origin outages for 1h. Write the header. (`public, max-age=30, s-maxage=300, stale-if-error=3600` + ETag.)

Topic 7.5 — Geo Routing

1. Why Interviewers Ask This

"How does a user in São Paulo hit the São Paulo PoP?" merges DNS (5.5), BGP/anycast, and product concerns (geo-blocking, data residency) — a synthesis question testing breadth.

2. Core Concept

Three layers of "geo" in production: 1. **Proximity routing** (performance): DNS-based geo/latency steering and/or anycast BGP — get the user to the nearest healthy PoP (mechanics in Module 5.5). 2. **Policy geo-routing** (product/legal): geo-blocking (licensing), data residency (EU data stays in EU), regional experiences — decided at edge by IP-geolocation databases. 3. **Origin-side geo** (architecture): multi-region origins; edge routes each user's *misses/writes* to the right home region (nearest, or their data's residency region).

3. Internal Working

- IP geolocation: commercial DBs (MaxMind et al.) mapping prefixes → country/city, ~99% country accuracy, much less at city level; edge annotates requests (`CF-IPCountry`) so backends never guess.
- Failure modes to name: VPNs/proxies, mobile CGNAT egress far from user, corporate egress in another country, IPv6 tunnel brokers — geo is *probabilistic*; legal-grade decisions need more than IP.
- Residency routing: user → nearest edge (always fine), but data operations pinned to home region: edge reads JWT claim/cookie/user-shard map → proxies to the correct regional origin over backbone.

4. Packet Flow Explanation

```
German user, EU-residency app, nearest edge = Frankfurt:
1. anycast -> FRA PoP (5ms)
2. edge worker: JWT says user.home=eu-central; static from FRA cache
3. API miss/write -> backbone -> eu-central origin only.
US user on vacation in Berlin:
1. same FRA edge (proximity is universal)
2. JWT home=us-east -> API proxied FRA->us-east over backbone
   (faster than user->us-east over public internet: warm conns, better path)
Geo-block case: CF-IPCountry=CU + policy -> edge returns 451 immediately;
origin never touched.
```

5. ASCII Diagram

```
proximity (perf):      user -> nearest PoP          (anycast/geo-DNS)
policy (legal):        edge checks IP-country -> allow/deny/variant
residency (data):      edge -> user's HOME region origin (claim/shard map)
three different "geo"s; conflating them = design smell.
accuracy: country ~99%, city ~55-80%, VPN/CGNAT = lies -> never use IP
geo for auth/security decisions alone.
```

6. Real Production Example

- Netflix licensing enforcement: IP-geo at edge + VPN-detection lists (an arms race worth mentioning).
- GDPR-era architectures: EU user shards pinned to EU regions with edge-routing by user-home claims.
- Game/launch traffic: geofenced staged rollouts by country at edge config.

7. Advantages

Edge geo decisions are fast (no origin hop for blocks), consistent (one enforcement point), and let a global anycast frontend coexist with regional/sovereign backends.

8. Trade-offs

- IP-geo is fuzzy → false blocks (VPN'd legitimate users, mislocated prefixes) = support burden; compliance regimes may require stronger residency proofs than IP.
- Cross-region proxying for travelers adds latency (correct trade for residency).
- Cache fragmentation if content varies by country (**Vary** -like key splits per geo — deliberate but costly).

9. Common Mistakes

- Using IP-geo for *authentication* decisions (“login from new country = block”) without treating it as a weak signal.
- Routing users to region by *request origin* instead of *data home* → user’s data “moves” when they travel (split-brain writes across regions!).
- Forgetting the CDN gives you the country header for free — teams re-deriving geo per service.

10. Performance Impact

Proximity: –50–200ms for far users. Backbone proxying for travelers: typically –20–40% vs public-internet direct (warm conns + private path) — i.e., residency routing costs less than intuition says. Geo-blocking at edge: zero origin cost for denied traffic (also DDoS-relevant).

11. Common Interview Questions

1. Design a global app with EU data residency — where does each request type get routed?
2. How accurate is IP geolocation and what breaks it?
3. Anycast vs GeoDNS for proximity — trade-offs (re-use Module 5.5 comparison).

12. Follow-up Questions

- “User travels from EU to US — what happens to their session/writes?” → still homed to EU region (claim-based routing); latency increases; discuss read-replica options.
- “How do you geo-block robustly against VPNs?” → you can’t fully; layered signals (IP lists, latency sanity checks, payment country) — say it’s an arms race, quantify business risk tolerance.

13. Debugging Scenarios

- Brazilian users intermittently routed to US PoP → their ISP’s peering/anycast path or geo-DNS resolver mislocation; verify with per-PoP header (**CF-Ray** suffix = PoP code) sampling by ASN.
- False geo-blocks after an IP-DB update → prefix reassignment lag; hold rollback lists, appeal path in product.

14. Best Practices

- Proximity by anycast/latency-DNS; policy at edge with the CDN’s country header; residency by *user-home claim*, never by request-source geo.
- Log PoP + country + origin-region per request for routing forensics; monitor per-country latency as SLO slices.

15. Practice Questions

1. Sketch request routing for a bank with: global static assets, EU+US customer shards, “EU data never leaves EU,” and travelers. Cover reads, writes, cache keys, and the traveler’s latency story.
 2. Your geo-blocked country’s traffic dropped only 70% after enabling edge blocking. Enumerate the leak paths. (VPN/proxies, mislocated prefixes, cached content already in-country ISP caches, direct-to-origin bypass — is origin locked to CDN?)
-

MODULE 7 — One-Page Cheat Sheet

ARCHITECTURE	user->edge PoP(hit ~10-30ms) ->shield ->origin(miss) consistent-hash cache nodes in PoP; request COALESCING; serve-stale = availability; dynamic traffic still wins (edge TLS + warm backbone + h3) ; lock origin to CDN only
EDGE COMPUTE	isolates ~0ms cold start; verify tokens/route/compose at edge; state: global-eventual KV vs single-home strong; staged rollouts (global blast radius!)
INVALIDATION	1) versioned URLs + immutable (static: never purge) 2) TTL + stale-while-revalidate (smooth) 3) surrogate-key/tag purge (~seconds global, SOFT purge) TTL backstop always; browsers are UNPURGEABLE -> short browser TTL / versioned URLs
HEADERS	no-store(never) vs no-cache(store, revalidate ALWAYS) max-age(browser) vs s-maxage(CDN) — split them! private public immutable SWR stale-if-error ETag->304; content-hash not inode; Vary: minimal (Vary: Cookie = death) ; Set-Cookie blocks caching
GEO	proximity(anycast/geo-DNS) != policy(IP-geo, fuzzy) != residency(user-home claim routing). country ~99%, city poor, VPN lies. edge country header for free.
NUMBERS	hit TTFB 10-30ms origin offload 90-99% purge ~150ms-s browser cache: infinite for immutable, else your max-age

MODULE 7 — Top Interview Questions

1. Full request walk through a CDN: hit, miss, shield, coalescing, headers consulted at each step.
2. Design news-site caching: full-HTML cache, corrections <10s (tag purge + SWR + soft purge).
3. The complete Cache-Control decision tree for four content classes.
4. Why does a CDN accelerate uncacheable APIs?
5. How do you handle a bad asset shipped with `immutable, max-age=1yr` ?
6. Thundering herd at TTL expiry / after purge — three defenses (coalescing, SWR, soft purge/jittered TTLs).
7. EU data residency with a global CDN — route reads/writes correctly.
8. `no-cache` vs `no-store` ; `max-age` vs `s-maxage` — with a production consequence for each confusion.

MODULE 7 — Common Mistakes

- Caching personalized responses (missing private/no-store; Set-Cookie).
- `Vary: Cookie / User-Agent` ; per-server ETags; query-string versioning.
- Purge-based consistency without TTL backstop; hard-purging hot keys.
- Believing you can purge browser caches.
- Leaving origin reachable directly (WAF/CDN bypass).
- One “geo” concept for what is really three (proximity/policy/residency).

MODULE 7 — Mock Interview (12 min)

Q1. “Black Friday: your product API origin died at 09:59; sale starts 10:00. CDN is in front. What do users see, and what should you have configured?” *Strong answer:* with `s-maxage + stale-if-error` + `serve-stale`: browsing keeps working from cache (prices possibly stale — business call), writes/checkout fail → queue/degrade gracefully; coalescing prevents miss-storms; static fallback page for true misses; postmortem items: SWR, SIE, origin autoscaling, load-shed order.

Q2. “Hit ratio is 38% on a site that’s 95% anonymous traffic. Find the money.” *Strong answer:* audit: `Vary` headers (Cookie?), Set-Cookie on HTML (analytics middleware!), cache-key includes marketing query params (`utm_*` — normalize them), TTLs too short, ETag mismatch across fleet blocking 304s, authenticated-path over-broad `private`. Expect 85–95% after fixes; quantify origin cost savings.

Q3. “Editor says a retracted article is still visible ‘for some people’ an hour after purge.” *Strong answer:* systematic: `Age / X-Cache` from multiple PoPs (one PoP missed purge → re-purge), response actually tagged? second cache layer (shield, service worker, browser — browser can’t be purged; was HTML TTL long?), corporate/ISP proxy; produce the layer-by-layer checklist and the config fix (short HTML browser TTL) so it can’t recur.